



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	RAZIA BANU
Student Name	P SHASHE PREETHAM
Roll Number	2520030534
Branch	CSE
Course Outcome	CO1

Case Study Topic: MediFlow Hospital Patient Index using BST & AVL Tree (Set 1 Q1)

Work Done Summary:

In this case study, both Binary Search Tree (BST) and AVL Tree data structures were implemented to manage and organize patient records using unique Patient IDs as keys. The Patient IDs were inserted according to the order in which appointments were registered, resulting in a nearly sorted sequence of data. Due to this insertion pattern, the BST gradually became skewed, resembling a linked list, which increased its height and reduced search efficiency. As a result, operations such as searching, insertion, and deletion took longer, approaching $O(n)$ time complexity in the worst case.

To address this limitation, an AVL Tree, which is a self-balancing binary search tree, was implemented. The AVL Tree automatically maintained balance after every insertion and deletion by performing appropriate rotations (left, right, left-right, and right-left). This ensured that the tree height remained minimal and balanced. A comparison was then made between the BST and AVL Tree in terms of tree height and operational performance. The analysis demonstrated that the AVL Tree consistently maintained a height proportional to $\log n$, enabling search, insertion, and deletion operations to be performed in $O(\log n)$ time. Therefore, AVL Trees proved to be more efficient and reliable for managing patient records in hospital management systems, where fast and consistent data retrieval is essential.

Implementation Details:

1. Inserted Patient IDs into BST and AVL Tree.
2. Observed BST degeneration due to sorted input.
3. Applied AVL rotations (LL, RR, LR, RL).
4. Compared tree heights and lookup performance.
5. Verified AVL maintained $O(\log n)$ operations.

Result: Screenshots / Output



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Coding\Projects\Case Studies> cd d:\tmp ; javac AVLPatientIndex.java ; java AVLPatientIndex
INSERTING PATIENT IDs
20 30 35 40 45 50 60 65 70 75 80 85 90

AVL Tree Inorder Traversal:
20 30 35 40 45 50 60 65 70 75 80 85 90
AVL Height = 4

Deleting 30...
Deleting 70...
Deleting 50...

Final AVL Tree Inorder:
20 35 40 45 60 65 75 80 85 90
Final AVL Height = 4
PS D:\tmp>
```

Result / Output

AVL Insertion Sequence

20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, 90

Patient IDs (Inorder Traversal)

20 30 35 40 45 50 60 65 70 75 80 85 90

AVL Height: 4

Final AVL Tree (Inorder Traversal After Deletions)

20 35 40 45 60 65 75 80 85 90

Final AVL Height: 4

Observations

- AVL Tree remained balanced after all insertions.
- Required rotations were performed automatically.
- Tree height remained logarithmic despite sorted input.
- Deletion of 30, 70, and 50 maintained AVL balance.
- Final AVL Tree continued to support efficient searching.

Time Complexity

- BST Worst Case Search/Insert/Delete: **$O(n)$**
- AVL Search: **$O(\log n)$**
- AVL Insert: **$O(\log n)$**
- AVL Delete: **$O(\log n)$**
- AVL Rotation (LL, RR, LR, RL): **$O(1)$**

GITHUB LINK : <https://github.com/P-Shashe-Preetham/DSA-CASE-STUDIES.git>

Student Signature	Faculty Signature

